



Recep Tayyip Erdoğan University

Faculty of Engineering and Architecture — Computer Engineering

CEN207 Data Structures (formerly CE205) — Syllabus

Fall Semester, 2026-2027

Course Information

Instructor	Asst. Prof. Dr. Uğur CORUH
Contact	ugur.coruh@erdogan.edu.tr — subject line must start with [CEN207]
Office	F-301
Office hours	By appointment by e-mail; meetings in the office or online with the university account
Lecture day, time, room	Friday 13:00–16:00 · İİBF & Faculty of Law Building, D-402 (ED-K4-2)
Course website	https://ucoruh.github.io/ce205-data-structures/
Course class	A new class is opened every term; the class code is announced in week 1
Language	English
Type / semester	Compulsory · 3 rd semester
Weekly hours / credit / ECTS	Theory 3 h · Credit 3 · ECTS 5
Prerequisite	CEN108 Algorithms and Programming II (former code CE100) — details: Prerequisites

A. Course Description

This course covers the fundamentals of data structures and file organization. It explains how data is mapped in programs — both in application run-time memory and in long-term file storage — and discusses the implementations, programming styles and run-time representations of these data objects. It also covers sorting, searching and graph algorithms. The aim

is to show how digital data structures solve real-world problems and how data is shaped and mapped to memory or storage. The course is practice-oriented: learning is strengthened through programming practice in class and a term project rather than theory alone.

B. Course Learning Outcomes

After completing this course, a student will be able to:

Code	Learning outcome
LO.1	Explain the definitions, representations and basic operations of fundamental linear (array, linked list, stack, queue) and non-linear (tree, graph) data structures.
LO.2	Analyze the time and space complexity of algorithms using asymptotic notation (Big-O) and compare the performance of different data structures.
LO.3	Implement fundamental sorting (insertion, selection, quick, heap) and searching (linear, binary) algorithms, and analyze and compare their performance.
LO.4	Implement balanced and unbalanced tree structures (binary trees, binary search trees, AVL trees, B-trees) and hash tables.
LO.5	Implement the graph data structure (representation methods) and fundamental graph algorithms (traversal, MST, shortest path).
LO.6	Explain sequential, direct (hash-based) and indexed sequential file organization techniques and evaluate their applications.
LO.7	Analyze a given problem, select the data structures and algorithms best suited to its requirements, and develop an efficient solution.

Contribution of learning outcomes to program outcomes (0–5)

	PO.1	PO.2	PO.3	PO.4	PO.5	PO.6–PO.12
LO.1	5	–	–	–	–	–
LO.2	5	–	–	–	–	–
LO.3	–	–	–	5	–	–
LO.4	–	4	3	5	–	–
LO.5	–	4	3	5	–	–
LO.6	–	–	–	4	5	–
LO.7	5	4	–	3	–	–

PO.1 Basic knowledge · PO.2 Problem solving · PO.3 Design · PO.4 Modern tools and techniques · PO.5 Research and experimentation.

C. Weekly Schedule

Rule for all assessments: **project demonstrations take place in the week just before the midterm and final exam weeks; quizzes take place inside the midterm and final exam weeks** so that every student can attend.

Week	Date	Topics	LO
1	18.09.2026 (make-up 23.09)	Course plan and communication. Introduction to linear and non-linear data structures; performance analysis (Big-O). Pointers and objects for data and variables; memory layout. Basics of ASN.1 / BER TLV / PER TLV. Intensive C workshop (toolchain, compile–run–debug).	1, 2, 7
2	25.09.2026	Linked lists (singly, doubly, circular, XOR) and skip lists; arrays (rotation, rearrangement, searching); matrices and sparse matrices.	1, 7
3	02.10.2026	Stacks (array and linked list, LIFO); expressions (infix, postfix, prefix) and conversions; queues (standard, circular, deque, multilevel; FIFO); Tower of Hanoi; recursion (groundwork for DFS).	1, 7
4	09.10.2026	Trees and binary trees; traversals (in-, pre-, post-order); heaps (min, max, binary, binomial, Fibonacci, leftist, k-ary) and priority queues; heap sort; Huffman coding.	1, 4, 7
5	16.10.2026	Graphs: representations (adjacency matrix, incidence matrix, adjacency list); traversals (BFS, DFS, iterative deepening, depth-limited, bidirectional); topological sorting; water jug problem.	1, 5, 7
6	23.10.2026	Searching (linear, binary, interpolation, Fibonacci); hashing and hash tables (direct-address tables, hash functions, chaining, open addressing, perfect hashing); collision resolution in practice.	3, 4, 7
7	30.10.2026	Midterm project demonstrations (C) and midterm project report.	1–5, 7
8	31.10– 08.11.2026	Midterm exam week — Quiz-1 (weeks 1–6).	1, 2, 4, 5, 7
9	13.11.2026	Graph algorithms: minimum spanning trees (Prim, Kruskal with disjoint sets), shortest paths (Dijkstra, Bellman–Ford), connectivity and SCC, maximum flow, cycle detection (Floyd, Brent), backtracking (n-queens, m-coloring, Euler and Hamiltonian paths).	3, 5, 7
10	20.11.2026	Sorting algorithms and taxonomy (insertion, selection, shell, quick, merge, heap, radix, counting, external sorting); comparison of sorting methods.	2, 3, 7
11	27.11.2026	Advanced trees: binary search trees, AVL, red-black, splay, B-tree family (2-3, 2-3-4, B+, B#), augmenting data structures; comparison of search trees.	4, 7
12	04.12.2026	Strings: string structures, search algorithms (brute force, Knuth–Morris–Pratt, Boyer–Moore, Horspool), LCS and edit distance (Levenshtein, Wagner–Fischer), alignment (Needleman–Wunsch, Smith–Waterman), tries and Patricia trees.	1, 3, 4, 7
13	11.12.2026	File organization I: sequential files (binary, interpolation, self-organizing search); direct files and hashing functions; collision	6, 7

Week	Date	Topics	LO
		resolution (coalesced hashing, progressive overflow, double hashing, buckets, Brent's method); perfect hashing.	
14	18.12.2026	File organization II: indexed sequential files; secondary key retrieval; binary and B-tree structures for files; hashing for expandable files (extendible, dynamic, linear hashing); k-d trees and grid files; external file sorting.	3, 4, 6, 7
15	25.12.2026	Final project demonstrations (Java) and final project report.	1–7
16	04–17.01.2027	Final exam period — Quiz-2 (weeks 9–14).	2–7

Enrichment topics kept from previous years and covered in the course notes as optional reading: alpha-beta pruning, Hasse diagrams, Petri nets, bipartite graphs, Bayesian networks, van Emde Boas trees, SimHash, trie hashing.

D. Textbooks, Software and Equipment

The course notes on the course website are the main resource and are self-contained. The following books are recommended for further reading:

- Deitel & Deitel. *C How to Program*, 7th ed. Prentice Hall, 2013.
- Y. Daniel Liang. *Introduction to Java Programming, Comprehensive Version*, 10th ed.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press.
- J. R. Hanly, E. B. Koffman. *Problem Solving and Program Design in C*, 6th ed.
- A. L. Tharp. *File Organization and Processing*. Wiley, 1988.
- P. Brass. *Advanced Data Structures*. Cambridge University Press, 2008.
- R. Sedgewick, K. Wayne. *Algorithms*, 4th ed. Addison-Wesley, 2011.

Laptop required. You will use your own development environment in class, in assignments and in the project: a C/C++ compiler (GCC, Clang or MSVC), CMake, GoogleTest, Doxygen, JDK 21 with Maven and JUnit 5, and Git with a GitHub account. Installation steps are given in week 1 and in the course notes; project templates are provided. See the [Prerequisites](#) page for the exact toolchain checklist.

E. Assessment

You carry out **one term project** with two checkpoints, each evaluated with its rubric: a midterm checkpoint (C implementation) and a final checkpoint (Java implementation). You also take one quiz in the midterm exam week and one quiz in the final exam period. The project topics, team-formation rules, deliverables and the **detailed midterm and final rubrics** (criteria, points, related learning outcomes and performance levels) are given in the course's [project guide](#).

Assessment	Code	Weight	When
Project checkpoint 1 — C implementation, report and demonstration (rubric)	RAP1	60% of midterm	Week 7 (30.10.2026)
Quiz-1 (weeks 1–6)	QUIZ1	40% of midterm	Week 8, midterm exam week (31.10–08.11.2026)
Project checkpoint 2 — Java implementation, report and demonstration (rubric)	RAP2	70% of final	Week 15 (25.12.2026)
Quiz-2 (weeks 9–14)	QUIZ2	30% of final	Week 16, final exam period (04–17.01.2027)

$$Grade_{Midterm} = 0.6 RAP1 + 0.4 QUIZ1 \quad Grade_{Final} = 0.7 RAP2 + 0.3 QUIZ2$$

$$Passing\ Grade = 0.4 Grade_{Midterm} + 0.6 Grade_{Final}$$

Workload (ECTS 5 = 125 hours)

Activity	Count	Hours	Total
Class attendance	14	3	42
Individual study (weekly notes and examples)	14	1	14
Quiz (midterm exam week and final exam period)	2	2	4
Individual study for quizzes	2	10	20
Project preparation (C and Java checkpoints)	2	16	32
Report preparation	2	5	10
Project presentation (demonstration and questions)	2	1.5	3
Total			125

F. Instructional Strategies and Methods

Lectures are face-to-face in the classroom and combine explanation, question–answer and hands-on programming. Each content week comes with course notes, slides, worked examples and self-check questions. Announcements, resources and submissions are handled in the course class. Attendance is taken.

G. Late Homework

Throughout the semester, assignments must be submitted by the announced deadline. Overdue assignments will not be accepted. Unexpected situations must be reported to the instructor by students.

H. Course Platform and Communication

All announcements, resources and submissions are shared in the course class, which is opened anew every term; the class code is announced in week 1. Course notes, slides and downloadable documents are on the course website. Check the class and your university e-mail every day.

I. Academic Integrity, Plagiarism & Cheating

Academic integrity is one of the most important principles of RTEÜ. Anyone who breaches the principles of academic honesty is severely punished.

It is natural to interact with classmates and others to "study together". It may also be the case where a student asks for help from someone else, paid or unpaid, to better understand a difficult topic or a whole course. However, what is the borderline between "studying together" or "taking private lessons" and "academic dishonesty"? When is it plagiarism, when is it cheating?

Looking at another student's paper or any source other than what is allowed during the exam is cheating and will be punished. However, many students come to university with very little experience of what is acceptable and what counts as "copying", especially for assignments. The following guidelines highlight the philosophy of academic honesty for graded assignments. If a situation arises that is not described below, ask the instructor whether what you intend to do stays within academic honesty.

a. What is acceptable when preparing an assignment?

- Communicating with classmates about the assignment to understand it better.
- Putting ideas, quotes, paragraphs or small pieces of code (snippets) found online or elsewhere into your assignment, provided that they are not themselves the whole solution and you cite their origin.
- Asking for help with the English language of your assignment.
- Sharing small pieces of your assignment in class to start a discussion on a controversial topic.
- Turning to the web or elsewhere for instructions, references and solutions to technical difficulties, but not for direct answers to the assignment.
- Discussing solutions with others using diagrams or summarized statements, but not actual text or code.
- Working with (even paying) a tutor, provided the tutor does not do your assignment for you.

b. What is not acceptable?

- Asking a classmate to see their solution before submitting your own.
- Failing to cite the origin of any text or code that you found outside the course and used in your work.
- Giving or showing your solution to a classmate who is struggling to solve the problem.

J. Expectations

You are expected to attend classes on time and complete the weekly requirements (readings and project milestones). The main communication channel between the instructor and students is e-mail. Send your questions from your university e-mail address; **include the course code in the subject line and your name in the message**. The instructor will also contact you by e-mail when necessary, so check your e-mail every day.

K. Lecture Content and Syllabus Updates

If deemed necessary, the lecture content or course schedule may change. Any change within the scope of this document will be announced by the instructor.